

# Invited: Graph Learning for Parameter Prediction of Quantum Approximate Optimization Algorithm

Zhiding Liang  
University of Notre Dame

Gang Liu  
University of Notre Dame

Zheyuan Liu  
University of Notre Dame

Jinglei Cheng  
Purdue University

Tianyi Hao  
University of Wisconsin - Madison

Kecheng Liu  
University of Notre Dame

Hang Ren  
University of California, Berkeley

Zhixin Song  
Georgia Institute of Technology

Ji Liu  
Argonne National Laboratory

Fanny Ye  
University of Notre Dame

Yiyu Shi  
University of Notre Dame

## ABSTRACT

In recent years, quantum computing has emerged as a transformative force in the field of combinatorial optimization, offering novel approaches to tackling complex problems that have long challenged classical computational methods. Among these, the Quantum Approximate Optimization Algorithm (QAOA) stands out for its potential to efficiently solve the Max-Cut problem, a quintessential example of combinatorial optimization. However, practical application faces challenges due to current limitations on quantum computational resource. Our work optimizes QAOA initialization, using Graph Neural Networks (GNN) as a warm-start technique. This sacrifices affordable computational resource on classical computer to reduce quantum computational resource overhead, enhancing QAOA's effectiveness. Experiments with various GNN architectures demonstrate the adaptability and stability of our framework, highlighting the synergy between quantum algorithms and machine learning. Our findings show GNN's potential in improving QAOA performance, opening new avenues for hybrid quantum-classical approaches in quantum computing and contributing to practical applications.

## KEYWORDS

QAOA, GNN, Max-Cut, Quantum Computing

## 1 INTRODUCTION

In the burgeoning field of quantum computing, particularly within the realm of Noisy Intermediate-Scale Quantum (NISQ) devices [7], Variational Quantum Algorithms (VQAs) have emerged as a promising avenue for harnessing quantum advantages in the near term. NISQ devices, characterized by their limited number of qubits and inherent noise, present a unique computational landscape. Despite these constraints, they offer a practical platform for early quantum

computations. The Quantum Approximate Optimization Algorithm (QAOA) is a prime example of algorithms tailored for NISQ machines [3], capitalizing on their capabilities to address complex combinatorial optimization problems like the Max-Cut problem.

NISQ devices are a key step in quantum technology, despite their limitations like shorter coherence times and higher error rates. Specialized algorithms like VQAs are developed to cope with these limitations. The synergy between NISQ hardware constraints and VQA algorithmic ingenuity is crucial in advancing quantum computing. VQAs, including QAOA, rely on PQCs, similar to quantum neural networks. The success of these algorithms depends on effective parameter initialization and optimization, addressing challenges like barren plateaus and local minima. Efficient initialization methods are vital for starting near a potential solution, enhancing optimization [1].

Recent trends in quantum computing have seen an intriguing amalgamation of classical and quantum learning architectures. Notably, the use of Graph Neural Networks (GNN) for solving combinatorial optimization problems has demonstrated promising results. GNNs, leveraging their ability to directly process graph structures, have shown remarkable success in diverse applications ranging from social network analysis to biological network modeling. This adaptability makes them particularly suited for quantum computing tasks, where many problems can be naturally represented as graphs. In this paper, we delve into this hybridization by exploring the use of GNNs for the initialization of the QAOA parameters. We posit that the integration of GNNs with quantum algorithms can significantly enhance the initialization process, particularly for complex problems like Max-Cut. This innovative approach opens up new avenues for leveraging the strengths of both machine learning and quantum computing to tackle some of the most challenging problems in computational science. Our focus is on harnessing the synergy between quantum computing and machine learning to tackle complex optimization tasks efficiently. Our main contributions are as follows:

- We introduce a novel initialization method for QAOA parameters using Graph Neural Networks (GNN). This approach leverages the strengths of both quantum computing and

---

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor, or affiliate of the United States government. As such, the United States government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for government purposes only. Request permissions from owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

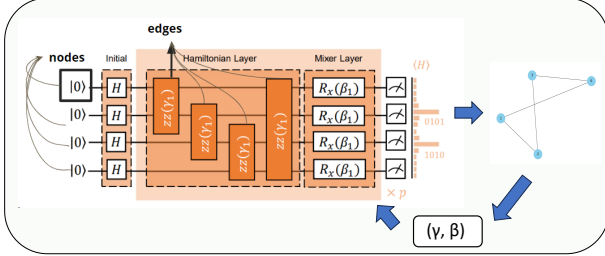
© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3663523>

machine learning. Our framework reduces the quantum resource overhead, making it more feasible for implementation on near-term quantum devices.

- We provide comprehensive benchmarking for different GNNs and analyze the suitable GNN for the QAOA case usage.



**Figure 1: The overview of the framework that uses the GNN to do the parameter “warm-start”.**

## 2 MOTIVATION

In our quest to harness the full potential of quantum computing, particularly for the Quantum Approximate Optimization Algorithm (QAOA), we recognize the power of Graph Neural Networks (GNN) as a pivotal tool. The current era of Noisy Intermediate-Scale Quantum (NISQ) devices presents unique challenges. Not only are quantum computing resources inherently limited on each device, but access to real quantum computers is also not readily available for most researchers. This limitation is compounded by the fact that access typically requires reliance on platforms provided by large corporations, leading to long wait times and substantial costs.

In pursuit of quantum computing’s potential, particularly for QAOA, we emphasize the synergy between GNN and QAOA, aiming to complement classical computers rather than replace them. Our approach utilizes affordable classical computational resources to simulate and identify optimal initial parameters for QAOA, addressing NISQ devices’ hardware limitations and making quantum computing more accessible and cost-effective. By optimizing algorithmic efficiency on classical computers before execution on quantum hardware, we aim to enhance QAOA’s practicality and effectiveness, unlocking new potentials in NISQ devices and advancing the field. Our approach aims to democratize access to quantum computing by reducing dependency on expensive platforms and maximizing preparatory work on classical systems.

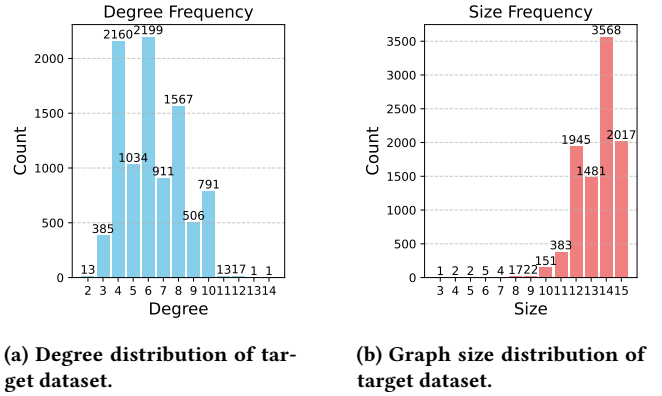
## 3 METHODOLOGY

Since no open-source dataset is available for our experiments, the first step of our methodology is data preprocessing. Then, in the later part, we will elaborate on model preparation and the detailed structure of each benchmark.

### 3.1 Data Processing:

We generate synthetic regular graphs comprising 9598 instances and simulate the parameters  $\gamma$  and  $\beta$  for the QAOA algorithm. These graphs vary in size, with nodes ranging from 2 to 15. The degree distribution of these graphs is captured in Figures 2a and

2b, illustrating that most graphs have a degree distribution ranging from 2 to 14, and node numbers primarily distributed from 3 to 15. The algorithm starts with randomly initialized values of  $\gamma$  and  $\beta$ , and then undergoes a process of optimization over 500 iterations. This optimization seeks to refine the values of  $\gamma$  and  $\beta$ , although the final values are optimized, they may not necessarily represent the absolute optimal parameters. In addition to determining the values of  $\gamma$  and  $\beta$ , the QAOA algorithm outputs solutions for the Max-Cut problem. It also provides an approximation ratio (AR) for these solutions compared to the optimal solutions derived from a brute-force search approach. We compute node degrees and one-hot encoding of node IDs as node features. The final output is an organized list comprising the graph structures along with important metadata like approximate ratio and values for the best cuts. This detailed dataset is then ready for further analysis and application in solving the Max-Cut problem using graph neural networks.



**Figure 2: Degree and graph size distributions of the target dataset.**

### 3.2 Model Preparation:

GNNs use the intrinsic graph structure and node features to learn a vector representation for each node or for the entire graph. The models update the representation of each node by aggregating feature information from local neighbors. Through iterative aggregation and combination of these features, GNNs can capture the topological structure of the graph within a node’s representation. Each iteration layer increases the receptive field of a node, allowing the incorporation of information from a wider neighborhood in the graph. For the center node  $v$  in a graph  $G$  and its neighbor nodes  $u \in \mathcal{N}(v)$ , the aggregation and combination function in GNNs is applied to the node representation  $\mathbf{h}_v^{(k-1)}$  at the  $(k-1)$  layer.

$$\mathbf{a}_v^{(k)} = \text{AGGREGATE}^{(k)} \left( \left\{ \mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v) \right\} \right), \quad (1)$$

$$\mathbf{h}_v^{(k)} = \text{COMBINE}^{(k)} \left( \mathbf{h}_v^{(k-1)}, \mathbf{a}_v^{(k)} \right), \quad (2)$$

where AGGREGATE and COMBINE are two functions crucial to the performance of GNN. Different choices result in different GNN models, such as GCN and GIN [5]. After stacking  $K$  layers, the readout function (e.g., an average function) for graph-level tasks is defined atop the node representations:

$$\mathbf{h}_G = \text{READOUT}(\{h_v^{(K)} | v \in G\}). \quad (3)$$

$\mathbf{h}_G$  is the graph representation, which we could use with an MLP for prediction. GNN encoders compress node features into a lower-dimensional space rich with both attribute and structural fidelity. Following a mean-pooling layer that amalgamates node embeddings, the prediction layer comes into play, forecasting critical graph-level properties such as the QAOA parameters  $\gamma$  and  $\beta$ . We aim to discern the impact of initial embeddings on the convergence efficiency and the overall solution quality, providing valuable insights into the optimization process within the QAOA landscape.

This comprehensive setup facilitates a meticulous comparison of each GNN architecture’s performance against a baseline of random parameter initialization. By doing so, we aim to discern the impact of initial embeddings on the convergence efficiency and the overall solution quality, providing valuable insights into the optimization process within the QAOA landscape.

### 3.3 Improvement of Data Quality:

In our preliminary construction of a basic Graph Neural Network (GNN) for research purposes, we encountered a significant challenge regarding the quality of our dataset. This dataset comprises 9585 graphs, generated by applying random initialization as the parameter initialization method for the Quantum Approximate Optimization Algorithm (QAOA). Unfortunately, this approach has resulted in many instances of low-quality data within our dataset. The GNN training targets are not optimal and have a relatively large gap compared to optimal solutions in many cases, which may be caused by noisy labels in the dataset. In particular, upon visualizing and analyzing this dataset, we observed that the approximation ratio for many groups of data was only around 50%. The primary cause of this issue can be attributed to the inherently complex optimization landscape of the QAOA algorithm. Random initialization may lead the optimizer into regions where not even local optima exist. This scenario severely undermines the performance of the algorithm, preventing it from achieving the desired optimization results. Consequently, this section of our study is dedicated to exploring more effective ways to enhance data quality, aiming to improve the reliability of the GNN in handling these challenges.

**Fixed Parameter Conjecture:** In our research, we utilized a method from work [9] involving fixed parameter sets (angles) for QAOA, optimized based on tree subgraphs. These fixed parameters are suggested as a universal solution for regular graphs, potentially simplifying and accelerating QAOA by removing the need for individual graph parameter optimization. The findings show that these fixed angles yield performance close to optimal, providing strong heuristic evidence for their effectiveness in regular Max-Cut graph QAOA. However, this method has a high computational cost. Despite searching in JPMorgan Chase’s quantum team’s open-source library [6], a leader in the quantum algorithm industry, we only found results for regular graphs with degrees ranging from 3 to 11, which constitute about 6% of our dataset, covering merely 587 graphs. This small subset’s improvement, while notable, was too insignificant to substantially enhance the performance of our Graph

Neural Network (GNN). The minimal impact of this method underscores the need for more comprehensive solutions to effectively improve GNN performance across the entire dataset.

**Selective Data Pruning:** In our advanced approach, the Selective Data Pruning (SDP) method was further refined to address the significant data quality issues in our quantum computing dataset. Recognizing that a substantial proportion of the dataset misdirected the GNN’s learning, we initially set an approximation ratio threshold of 70%, pruning data below this mark. However, this approach, while improving the overall data quality, led to a considerable loss of data. The reduced dataset size was insufficient for the GNN to adequately learn across the entire design space of the parameter space, hindering its ability to generalize and effectively model diverse quantum computing scenarios.

To address this issue, we introduced a selective rate, allowing us to fine-tune the balance between retaining and pruning poor-quality data. For instance, setting a selective rate of 70% would mean preserving 70% of the otherwise discarded data, while pruning the remaining 30%. This nuanced approach provided a more balanced dataset, retaining enough data diversity for effective learning while still ensuring that the data quality was high enough to guide the GNN towards meaningful patterns and relationships. The introduction of the selective rate transformed the SDP method into a more dynamic and adaptable tool. It enabled us to iteratively refine our dataset, continuously assessing the impact of different selective rates on the GNN’s performance.

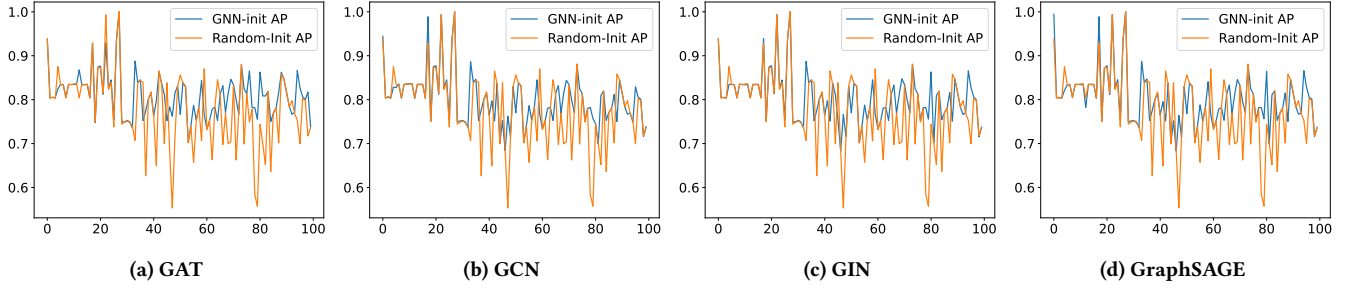
## 4 EXPERIMENTS

In this section, we conduct experiments to validate the effectiveness of graph benchmarks in solving QAOA problem under fixed parameters setting.

### 4.1 Experiment Setup

**Dataset and Baseline Models.** We set aside 100 test graphs with different degrees and graph sizes to calculate the improvement in the approximation ratio achieved by different GNN-based QAOA initialisation. In particular, we focus on four GNN benchmarks: GCN, GAT, GIN and GraphSAGE and one baseline - random initialisation. The results for these 100 instances are presented in Figure 3, where the orange line is the approximation ratio for random initialisation and the blue line the approximation ratio for various GNNs. The break down results are presented in Table 1.

**Implementation Details.** We report the mean and standard deviation of the result across 100 test graphs. For each GNN model, we set the input dimension to 15, number of GNN layer to 2. Furthermore, we set embedding dimension to 32 and dropout ratio to 0.5 during training, which ensures that no single neuron becomes overly specialized to rely on specific features of the input data. This encourages the network to learn more robust features that generalize better to unseen data. We use Adam optimizer to optimize the model and ReduceLROnPlateau as scheduler to monitor the training loss and reduces the learning rate when there is no improvements for a defined number of epochs. In particular, we set scheduler mode to min, factor to 5, patience to 5 and minimum learning rate to  $1e-5$ . Lastly, we train each model for 100 epoches before examine it on test set.



**Figure 3: Comparison of Approximation Ratio between random initialisation and various GNN benchmarks. From left to right, the order is GAT, GCN, GIN and GraphSAGE.**

| Methods             | GAT             | GCN              | GIN             | GraphSAGE        |
|---------------------|-----------------|------------------|-----------------|------------------|
| Average Improvement | $3.28 \pm 9.99$ | $3.65 \pm 10.17$ | $3.66 \pm 9.97$ | $2.86 \pm 10.01$ |

**Table 1: Average improvements of GNN benchmarks compared to random initialization.**

## 4.2 Result Analysis

In our experiment comparing various Graph Neural Network (GNN) benchmarks for initializing the Quantum Approximate Optimization Algorithm (QAOA) against random initialization, the results showed varied performance across different GNN architectures. The baseline, random initialization, serves as a comparison point. Graph Attention Networks (GAT) slightly outperformed compared to random initialization, with a score of  $3.28 \pm 9.99$ . Graph Convolutional Networks (GCN) improved over the baseline, scoring  $3.65 \pm 10.17$ , indicating that convolutional approaches in GCN might be capturing useful features for QAOA, but not to a large extent. Graph Isomorphism Networks (GIN) showed slightly better performance with  $3.66 \pm 9.97$ , potentially due to their ability to capture structural information in graphs. In contrast, GraphSAGE, with a score of  $2.86 \pm 10.01$ , improved only a bit in terms of performance compared to random initialization, denoting that its inductive learning approach might not align well with QAOA initialization requirements. And from Figure 3, we can see that across test graphs, the performance of GNN benchmarks is more stable than the random initialization approach. For example, the Graph Isomorphism Networks (GIN) show a relatively more stable performance with fewer instances where random initialization surpasses GNN initialization, suggesting a potentially more reliable performance.

## 5 RELATED WORK

[2] using Goemans-Williamson random rounding to warm-start recursive QAOA and shown a consistent improvement in the cut size for fully connected graphs with random weights for the Max-Cut problem. This approach can also be applied to other random rounding schemes and optimization problems. And recent work [8] introduce a classical pre-processing step that initializes QAOA with a biased superposition of possible cuts, referred to as a warm-start. A close work [4] use GCN and line graph neural network (LGNN) for warm start the QAOA. In their framework, GCN and LGNN

update graph embeddings based on neighbors using a function  $f\theta$ , outputting probabilities for each node’s cut side.

## 6 FUTURE WORKS AND LIMITATIONS

In this project, we identified challenges in three areas: problem definition, data quality, and GNN structure suitability for QAOA. Firstly, the focus on unweighted graphs limits the model’s applicability, necessitating a redefinition to include weighted graphs. Secondly, data quality issues, particularly noise levels, complicate training and impact accuracy. Lastly, current GNN structures may not be optimized for QAOA. Future work should aim to redefine the problem to cover both weighted and unweighted graphs, improve data preprocessing methods, and develop advanced GNN architectures tailored for QAOA to create a more versatile and applicable model for quantum optimization tasks.

## 7 ACKNOWLEDGMENT

This material is supported by the DOE-SC Office of Advanced Scientific Computing Research AIDE-QC project under contract number DE-AC02-06CH11357.

## REFERENCES

- [1] Jinglei Cheng, Haoqing Deng, and Xuehai Qian. 2020. Accqoc: accelerating quantum optimal control based pulse generation. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Valencia, Spain, 543–555.
- [2] Daniel J Egger, Jakub Mareček, and Stefan Woerner. 2021. Warm-starting quantum optimization. *Quantum*, 5, 479.
- [3] Gian Giacomo Guerreschi and Anne Y. Matsuura. 2019. Qaoa for max-cut requires hundreds of qubits for quantum speed-up. *Scientific reports*, 9, 1, 1–7.
- [4] Nishant Jain, Brian Coyle, Elham Kashefi, and Niraj Kumar. 2022. Graph neural network initialisation of quantum approximate optimisation. *Quantum*, 6, 861.
- [5] Gang Liu, Tong Zhao, Eric Inae, Tengfei Luo, and Meng Jiang. 2023. Semi-supervised graph imbalanced regression. *arXiv preprint arXiv:2305.12087*.
- [6] Danylo Lykov, Ruslan Shaydulin, Yue Sun, Yuri Alexeev, and Marco Pistoia. 2023. Fast simulation of high-depth qaoa circuits. In *Proceedings of the SC’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, 1443–1451.
- [7] John Preskill. 2018. Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79. arxiv:1801.00862.
- [8] Reuben Tate, Majid Farhadi, Creston Herold, Greg Mohler, and Swati Gupta. 2023. Bridging classical and quantum with sdp initialized warm-starts for qaoa. *ACM Transactions on Quantum Computing*, 4, 2, 1–39.
- [9] Jonathan Wurtz and Danylo Lykov. 2021. Fixed-angle conjectures for the quantum approximate optimization algorithm on regular maxcut graphs. *Physical Review A*, 104, 5, 052419.